Claude Code work coordination: Projects and dynamic execution workflows

Strengths and best use

Claude Code Projects and dynamic workflows address different parts of sustained work. A Project maintains a coordinating conversation, shared instructions, memory, files, and task threads. Dynamic workflows organize the execution of a difficult task. Together they can reduce repeated context setup while keeping individual deliverables reviewable. Projects (https://code.claude.com/docs/en/claude-projects) - Dynamic workflows (https://claude.com/resources/articles/introducing-dynamic-workflows-in-claude-code)

Verify availability first

As checked November 7, 2026, Projects documentation describes a gradual public beta for Pro, Max, and Ultra, with Team and Enterprise unavailable for this feature. Check Projects or More in the sidebar and the documented prerequisites for the account. Cloud repository threads need the Claude GitHub App with appropriate repository access. A connection established only through `/web-setup` is insufficient for those threads.

Dynamic workflows are documented separately as available on paid plans and through supported platform surfaces. The article describes asking for a dynamic workflow or using the available high-effort controls and `ultracode`. Availability of one feature does not establish access to the other. Read the current interface and usage details before budgeting a long run. Common workflows (https://code.claude.com/docs/en/workflows)

Set up one stream of work

Choose a goal that outlasts one session: a migration, a service-quality target, or a recurring research area. Add only relevant repositories and files. State acceptance criteria, review expectations, network needs, secrets handling, and boundaries. The coordinator sees thread reports rather than every intermediate step; useful handoffs need concrete evidence and unresolved questions.

Cloud threads receive project context and repository CLAUDE.md files, but the current Projects documentation says repository skills, `.claude/settings.json`, and `.mcp.json` are not automatically included. Verify the cloud environment and connected tools instead of assuming local settings follow the task. Tasks needing a local database, emulator, or private network may require local execution.

Execute and evaluate

For a bounded pilot, select one issue with a reproducible failure. Ask for a plan and an explicit review checkpoint, then inspect the resulting change and its evidence. Record elapsed time, usage, retries, handoff failures, and review effort. Parallel work increases the importance of ownership and dependency order. Do not infer that more agents or tokens improve every task. Persistent coordination is most valuable when new tasks keep arriving; a single-session issue may need only a normal session.

Reusable prompt

```
Scope this work into a persistent project context and one bounded execution task. State acceptance criteria, ownership, environment needs, and review checkpoints. Check which local settings and tools are available in the chosen execution surface. Return concrete results, verification evidence, usage observations, and unresolved decisions to the coordinator.
```

Project idea

Compare one representative issue handled in a normal session with a similar issue using a Project and a dynamic workflow. Use comparable acceptance criteria and include the human's review time in the result.

Original sources

- https://youtu.be/omihvjf0A2k
- https://www.instagram.com/reel/DZLdAqhTO2X/